Author: Lev Popov 2521041 | Course: Software Testing & QA | VUM

Assignment: Final Project – Automated UI Testing with Selenium & Pytest

## 1. Project Overview

The objective of this project is to build an automated UI test suite for the **Photo Tech World** web application (index.html / script.js). The suite validates all key user flows: login, navigation between tabs, contact-form field validation, the Send Message button, and the Logout button.

All tests are implemented in **Python 3**, executed with the **pytest** framework, and driven by **Selenium WebDriver** against **Google Chrome**.

## 2. Technologies Used

| Component | Tool / Library | Purpose |
|---|---|---|
| Language | Python 3.x | Test implementation language |
| Test Framework | pytest | Test discovery, fixtures, assertions, reporting |
| UI Automation | Selenium WebDriver | Browser control and DOM interaction |
| Browser | Google Chrome + ChromeDriver | Target browser (mandatory per spec) |
| Browser Options | selenium.webdriver.chrome.options | Headless mode, window size |
| Wait Strategy | WebDriverWait + EC | Explicit waits for dynamic elements |

## 3. Project File Structure

The test project contains a single file as required by the assignment:

- **st2521041_py_test.py** – all pytest test cases
- **st2521041_py_doc.pdf** – this documentation file
- **st2521041_py_present.pdf** – slide presentation for oral defence

The HTML/JS source files (index.html, script.js, style.css) are provided separately and must be placed in a known local path before running the tests.

## 4. WebDriver Fixture Design

A module-scoped pytest fixture called **driver** creates a single Chrome WebDriver instance shared across all tests in the module. This avoids repeatedly opening and closing the browser, reducing total execution time.

Key Chrome options configured:

- **--headless** – run without a visible browser window (CI-friendly) • **--no-sandbox** – required in restricted Linux environments • **--disable-dev-shm-usage** – prevents shared-memory errors in containers • **--window-size=1280,800** – consistent viewport for layout-dependent tests

A helper function login(driver, username, password) centralises the login action and is called from fixtures that require an authenticated session.

## 5. Test Classes and Coverage

### TestLogin (5 tests)

• Login page is visible on initial load • Invalid credentials display an error message • Valid credentials hide the login page • Valid credentials show the main app • User initials (AD) are displayed in the top bar

### TestNavigation (5 tests)

- Cameras section is active directly after login
- Clicking Lenses shows the Lenses section
- Clicking Contact shows the Contact section
- Navigating back to Cameras works correctly
- Only one page section is visible at a time

### TestContactForm (8 tests)

- Empty name triggers 'Name is required' error on submit
- Filled name clears the name error
- Empty email triggers 'Email is required' error
- Malformed email triggers format error
- Valid email clears the email error
- Empty phone triggers 'Phone is required' error
- Too-short phone triggers digit-count error
- Valid phone clears the phone error

### TestSendMessage (2 tests)

- Send Message does not show success when fields are empty
- Send Message shows 'Message sent successfully!' with valid data

### TestLogout (1 test)

- Clicking Logout reloads the page and shows the login screen

## 6. Validation Rules Tested (from script.js)

| Field | Rule | Error Message |
|---|---|---|
| Name | Must not be empty (trimmed) | Name is required |
| Email | Must not be empty | Email is required |
| Email | Must match /^[^\s@]+@[^\s@]+\.[^\s@]+$/ | Email must follow example@mail.com |
| Phone | Must not be empty | Phone is required |
| Phone | Must match /^\+?[0-9\s]{7,15}$/ | Phone must contain 7-15 digits |

## 7. How to Run the Tests

Prerequisites:

- Python 3.x installed
- Google Chrome browser installed
- ChromeDriver matching the installed Chrome version on PATH
- Dependencies: pip install pytest selenium

Before running, update the file path in the driver fixture:

```
drv.get("file:///absolute/path/to/index.html")
```

Run all tests:

```
pytest st2521041_py_test.py -v
```

Run a specific class:

```
pytest st2521041_py_test.py::TestLogin -v
```

## 8. Known Issue in Source Code

A bug exists in **script.js** inside the submitForm() function. An event listener is added to a non-existent element #myForm on every form submission, which throws a TypeError at runtime. This does not prevent the success message from appearing because the error occurs after the success check, but it pollutes the browser console. The tests work around this by interacting via the actual <form> submit button rather than the missing element.

## 9. Locator Strategy

Elements are located using the following Selenium strategies, chosen in order of stability:

| Strategy | Used For | Example |
|---|---|---|
| **By.ID** | Inputs, error/success divs, page sections | find_element(By.ID, "email") |
| **By.XPATH** | Buttons (no ID), nav links, submit button | find_element(By.XPATH, "//nav//button[text()='Lenses']") |
| **JS exec** | Triggering inline JS validation functions | execute_script("validateEmail()") |

ID-based locators are preferred for stability. XPath text-matching is used for buttons that have no unique ID. JavaScript execution is used for the inline validation functions (validateEmail(), validatePhone()) because they are bound to the oninput event rather than individual button clicks.